

# OS Assignment 2

Luiba Javed  
24K-0014

Date:

M T W T F S S

## Chap 4 - Homework

- 4.1 :-
- 1) Web Servers : Handling multiple client requests simultaneously
  - 2) GUIs : Staying responsive while a background thread fetches data
  - 3) Video Rendering: Dividing frames or pixels across thread to process them in parallel.

4.2 - 60% parallel,  $s = 0.40$

(a) 2 cores:  $\text{Speedup} = \frac{1}{0.4 + \frac{0.6}{2}} = \frac{1}{0.7} \approx 1.43x$

(b) 4 cores:  $\text{Speedup} = \frac{1}{0.4 + \frac{0.6}{4}} = \frac{1}{0.55} \approx 1.82x$

4.3 Task parallelism - Each thread is performing a unique task rather than performing the same operation on pieces of a single large dataset

4.4 , User threads are managed by a library & kernel threads are managed by OS

2) If user threads blocks, the whole process blocks but if a kernel thread blocks, the OS can schedule another thread.

→ User threads are better for fast context switching & kernel threads are better for utilizing multicore processors

4.5 The kernel saves the current thread's register set & PC, then loads the saved registers & PC of the next thread to be executed

4.6 Thread: minimal resources (Stack, Registers, Program Counter)

Process: Heavy resources (Memory address space, file, Heap & Data Segment)

4.7 Yes, realtime threads require guaranteed CPU time. Binding to a LWP ensures that the kernel-level scheduler can manage its priority directly



4.8 1) Serial Tasks: A program where each step strictly depends on the result of the previous one.

2) Single-core CPU-bound tasks: The overhead of switching between threads can actually make the program slower than running it linearly

4.9 When threads are I/O bound. While one thread waits for I/O, the kernel can switch to another thread to keep the CPU busy

4.10 b. Heap memory c. Global variables  
registers on stack are private to each thread

4.11 No, if the kernel only sees the process as a single entity, it will only schedule it on one core, regardless of how many user thread exists.

4.12 No, using separate processes provides fault tolerance & security isolation.

1) a thread crashes in a multithreaded browser, the whole browser crashes

4.13 Yes, on a single-core system, a scheduler can interleave execution, but since only one instruction runs at a time, there is no true parallelism.

4.14 Adhmal's Law Calculations:

40% parallel ( $S = 0.6$ ) → (a) 8 cores:  $1.54x$ , (b) 16 cores:  $1.60x$

67% parallel ( $S = 0.3$ ) → (a) 2 cores:  $1.50x$ , (b) 4 cores:  $2.01x$

90% parallel ( $S = 0.1$ ) → (a) 4 cores:  $3.08x$ , (b) 8 cores:  $4.71x$

4.15 • Thumbnails - Data • Network read/write - Task • Grand central  
• Matrix Transpose - Data • Array summation - Data • Dispatch - Task



4-16 I/O - 1 thread since it uses a single file, adding more threads to one disk often adds overhead

CPU - 4 threads to match 4 available processing cores

4-17 a) Processes : 6 unique processes

b) Threads : 2 unique threads created by thread\_create

4-18 LINE C(child) : value = 5

LINE P(Parent) : value = 0 - child change does not affect the parent

4-18 In linux, processes & threads are separate. Both are treated as tasks & created using clone() sys call. The behavior depends on the flags passed whether resources like memory are shared or not, provides high flexibility but less intuitive

In contrast, windows clearly separates processes & threads. A process acts as a container with its own memory, & it contains one or more threads that execute within it. The model is easier to understand & manage but less flexible

4-22 & 4-23 are codes - submitted separately.



## Q5 Review Of Research Paper

### Introduction

The research paper discusses how to improve the performance of string matching algos. which are commonly used in areas like search engines, bioinformatics & data processing. With the rapid growth of large databases, traditional algos take a lot of time to process data. To solve this problem, the authors propose a parallel version of the E-Attheer algo using P-threads. The main idea is to use multiple cores of a processor to perform tasks simultaneously & reduce execution time.

### Objectives Of the Study

The main goal of this research is to enhance the performance of the E-Attheer Algo, reduce the time required for string matching & Analyze performance using diff datasets, pattern lengths & number of threads.

### Methodology

Authors used parallel processing with Pthreads to implement the algo. Instead of processing the entire data sequentially, the input text is divided into smaller part. Each part is then assigned to a separate thread, & all threads run in parallel. Key steps include:

- dividing the data among threads then running the same pattern matching process on each chunk, then using mutex locks to avoid conflicts when threads share data & then combining the results from all threads at the end.

Performance is measured using execution time & Speedup

### Results

The results clearly shows that the parallel version performs much better than the sequential version. Some imp observations were: The pitch dataset gave the best execution time. 2) DNA dataset achieved the highest speedup. 3) XML & Protein Datasets showed lower performance in some cases. It was also observed that increasing number of cores improves performance. However too many cores can cause overhead & communication delays, which reduce efficiency.



### Strengths of the Paper

- The paper addresses a real-world problem of handling large datasets
- It effectively uses parallel computing techniques
- The approach is practical for modern multi-core systems.
- Clearly demonstrates improvement in execution time & speedup.

### Limitations

Performance is not consistent across all datasets & overhead increases when more threads are used. The approach is limited to CPU-based systems only.

### Critical Analysis

The paper shows that parallel processing improves string matching efficiency. Pthreads work well for shared memory systems but may not scale to larger or distributed environments. Performance also varies with the dataset & comparison with modern parallel methods could have strengthened the study. Future improvements could include better thread management & optimization techniques.

### Conclusion

In conclusion, the paper improves the E-Atter Algo using parallel processing with Pthreads, achieving better performance than sequential approach. Despite some limitations, it provides good foundation for future research.